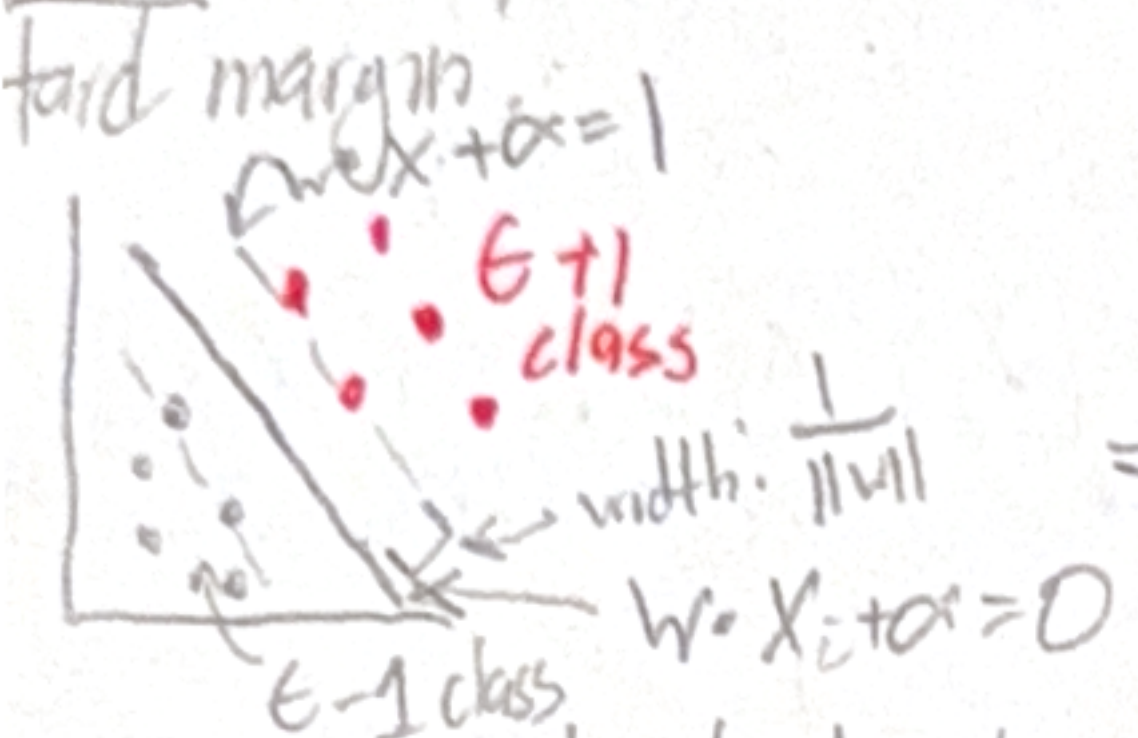


Gradient Method

Compute mean μ of each class $\{x_i\}$
 $f(x) = (\mu_i - \mu_x) \cdot x - (\mu_i - \mu_x) \cdot \frac{\mu_i + \mu_x}{2}$
 Decision boundary is the hyperplane that bisects line segment between μ_i, μ_x .

SVMs



$$y_i (w \cdot x_i + \alpha) \geq 1$$

$$\frac{1}{||w||} |w \cdot x_i + \alpha| \geq \frac{1}{||w||}$$

$$\Rightarrow \min ||w||^2 \text{ s.t. } y_i (x_i \cdot w + \alpha) \geq 1$$

At the optimal sol'n, the margin is exactly $\frac{1}{||w||}$. At least 1 constraint holds w/ equality.

Kernel: Instead of $w \cdot x + \alpha = 0$, use $w \cdot \Phi(x) + \alpha = 0$, where Φ is a vector of nonlinear functions of x .

Descent Algs

GD: $w \leftarrow w - \epsilon \nabla f(w)$

SGD: Loss function $f(w) = \sum f_i(w)$

Choose random i and update w/

$$w \leftarrow w - \epsilon \nabla f_i(w)$$

Newton's:

$$w \leftarrow w - (\nabla^2 f(w))^{-1} (\nabla f(w))$$

Or, for better efficiency,

$$e \leftarrow \text{sol'n to } (\nabla^2 f(w))e = -\nabla f(w)$$

$$w \leftarrow w + e$$

QDA - Assume all classes have the same Σ . Fit a Gaussian to each class, choose the class w/ the highest posterior. Yields a linear boundary.

$$\ln(\pi_c) - \frac{1}{2} (x - \mu_c)^T \Sigma^{-1} (x - \mu_c)$$

$$\ln(\pi_c) - \frac{1}{2} \ln(|\Sigma|) - \frac{1}{2} (x - \mu_c)^T \Sigma^{-1} (x - \mu_c)$$

$$= \ln(\pi_c) + x^T \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^T \Sigma^{-1} \mu_c - \frac{1}{2} x^T \Sigma^{-1} x$$

For a univariate Gaussian, choose the class that maximizes

$$\frac{\mu_c \cdot x}{\sigma^2} - \frac{||\mu_c||^2}{2\sigma^2} + \ln(\pi_c)$$

Isocenters for $x^T M x > 0 \forall x \Leftrightarrow \lambda_i > 0$

Isocenters for $x^T M x \geq 0 \forall x \Leftrightarrow \lambda_i \geq 0$ $q_1(z) = ||z||^2$

Indef: +ve eigenvalue +ve eigenvalue - Gaussians are monotonic transforms of $\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu)$ (share the same isocenters)

2D: bowl $\cup$ Variance/Covariance

3D: valley $\cup$ $\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$; $\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$

Indef: saddle $\cap$ $\text{Cov}(R, S) = E[(R - E[R])(S - E[S])^T] = E[RS^T] - E[R]E[S]^T = E[XY] - E[X]E[Y]$

Perceptron Algorithm (gives linear boundary)

Find weights w s.t. $x_i \cdot w \geq 0$ if $y = 1$ and $x_i \cdot w < 0$ if $y = -1$

$$\text{Loss } L(z, y_i) = \begin{cases} 0 & \text{if } y_i z \geq 0 \\ -y_i z & \text{otherwise} \end{cases}$$

run descent algo to solve

$$R(w) = \frac{1}{n} \sum_{i \in V} -y_i x_i \cdot w$$

$V = \text{set of indices where } y_i x_i \cdot w < 0$

Soft margin: Allow some points to violate the margin w/ slack variables

$$\min ||w||^2 + C \sum \epsilon_i$$

$$\text{s.t. } y_i (x_i \cdot w + \alpha) \geq 1 - \epsilon_i$$

$$\epsilon_i \geq 0$$

Small hyperparam C : maximal margin, underfit, less loose boundaries, more flat w/ kernel

Loss function $L(z, y)$ specifies badness if classifier predicts z and the true class is y .

Risk: expected loss

$$\text{risk} = E[L(g(x), Y)]$$

$$= \sum_x (L(g(x), 1) P(Y=1|X=x) + L(g(x), -1) P(Y=-1|X=x)) P(X=x)$$

Bayes' Rule: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$

$$P(Y=1|X) = \frac{P(X|Y=1)P(Y=1)}{P(X)}$$

Bayes Optimal Decision boundary.

When L is symmetric, choose the class w/ the largest posterior prob.

$$r^*(x) = \arg \min_i L(i, 1) P(Y=1|x) + L(i, 2) P(Y=2|x)$$

QDA (decision boundary is quadric - multivariate generalization of quadratic)

Different covariance matrix for each class.

Maximize the log of the posterior:

$$Q_c(x) = \ln(\pi_c) - \frac{1}{2} \ln(|\Sigma_c|) - \frac{1}{2} (x - \mu_c)^T \Sigma_c^{-1} (x - \mu_c)$$

Eigenstuff $Av = \lambda v$

$$\text{Spectral thm: } A \in \mathbb{R}^n \Rightarrow A = U^T \Lambda U$$

orthonormal eigenvectors

Contours of $x^T M x$ are ellipsoids determined by eigenvectors/values of $M^{-1/2}$

Spectral case: M diagonal $\rightarrow$ contours are axis aligned.

Contours of $x^T M x$ are ellipsoids determined by eigenvectors/values of $M^{-1/2}$

Spectral case: M diagonal $\rightarrow$ contours are axis aligned.

Contours of $x^T M x$ are ellipsoids determined by eigenvectors/values of $M^{-1/2}$

Spectral case: M diagonal $\rightarrow$ contours are axis aligned.

Contours of $x^T M x$ are ellipsoids determined by eigenvectors/values of $M^{-1/2}$

variance / Variance cov.

$$\text{Var}(R) = \begin{bmatrix} \text{Var}(R_1) & \dots & \text{Cov}(R_1, R_d) \\ \vdots & \ddots & \vdots \\ \text{Cov}(R_d, R_1) & \dots & \text{Var}(R_d) \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} \text{Cov}(R_1, R_1) & \dots & \text{Cov}(R_1, R_n) \\ \vdots & \ddots & \vdots \\ \text{Cov}(R_n, R_1) & \dots & \text{Var}(R_n) \end{bmatrix}$$

$$\hat{\Sigma} = \frac{1}{n} \sum (x_i - \hat{\mu})(x_i - \hat{\mu})^T$$

R_i, R_j independent $\Rightarrow \text{Cov}(R_i, R_j) = 0$ (not the other way around)
 All features pairwise ind. $\Rightarrow \Sigma$ is diagonal
 $\text{Var}(R)$ diagonal + joint normal $\Leftrightarrow$ axis-aligned Gaussian, squared radii
 and diagonal of $\Sigma = \text{Var}(R) \Leftrightarrow f(x) = f(x_1)f(x_2)\dots f(x_d)$
 $\Sigma \succeq 0$ for Gaussians
 $\Sigma \succeq 0$ in general
 $\hat{\Sigma} = \frac{1}{n} \sum (x_i - \hat{\mu})(x_i - \hat{\mu})^T$
 sample cov. $\hat{\Sigma} = \frac{1}{n} X^T X$
 For zero-mean data, $\hat{\Sigma} = \frac{1}{n} X^T X$

Regression: loss function + regression function + cost function Least squares polynomial regression.
Least squares: $\arg\min_w \|Xw - y\|^2 = (X^T X)^{-1} X^T y = w^*$ Run least squares on feature matrix
 $\Phi(x_i) = [x_{i1}^2, x_{i1}x_{i2}, x_{i2}^2, x_{i1}x_{i2}]$
 - Add "feature dimension" of all 1's to X to get intercept term
 - Sensitive to outliers since residuals are squared

Logistic regression: fits probabilities in range $(0, 1)$ Weighted least squares
 - Assign each sample pt. a weight (importance) w_i , collect w_i 's in Ω .

$$s(x) = \frac{1}{1 + e^{-x}}, s'(x) = s(x)(1 - s(x))$$

$$w^* = \arg\min_w \sum_i L(s(x_i; w), y_i)$$

$w^* = \arg\min_w \sum_i L(s(x_i; w), y_i)$ Lib. sep. $\Rightarrow w^* \rightarrow \infty$
 $\rightarrow$ yields linear boundary - Modes log odds as linear
 - Penalizes large weights
 - Equivalent to assigning a Normal prior on w : $w \sim N(0, \sigma^2)$
Ridge Regression

$$w^* = \arg\min_w \|Xw - y\|^2 + \lambda \|w'\|^2 = (X^T X + \lambda I)^{-1} X^T y$$

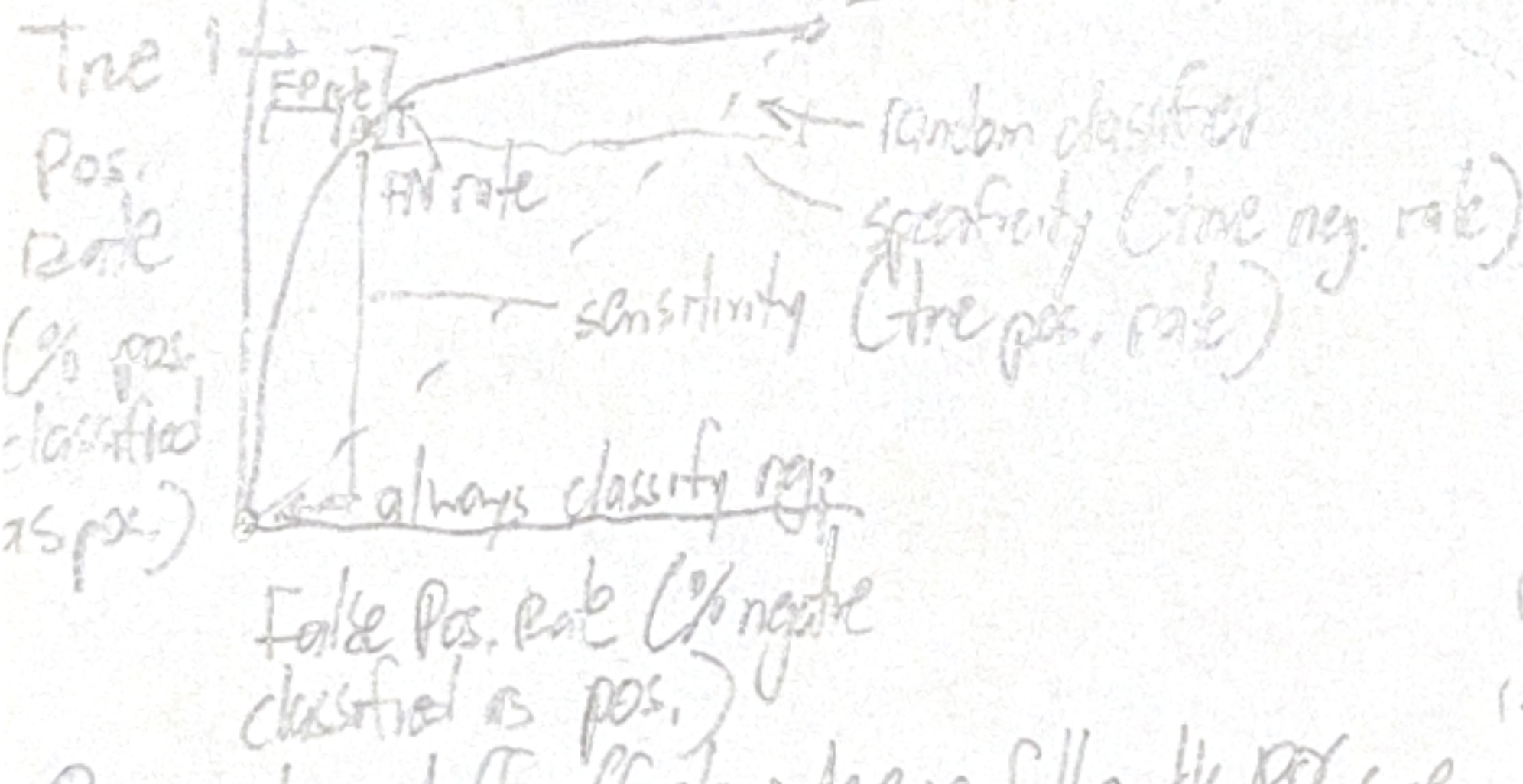
$$w^* = \arg\min_w \sum_i y_i \ln(s(x_i; w)) + (1 - y_i) \ln(1 - s(x_i; w))$$

$\nabla_w J = -X^T (y - s(Xw))$ where $s(Xw) = \begin{bmatrix} s(x_1; w) \\ \vdots \\ s(x_n; w) \end{bmatrix}$ LASSO

$$\nabla_w^2 J = X^T \text{diag}(s_i(1 - s_i)) X$$

$$w^* = \arg\min_w \|Xw - y\|^2 + \lambda \|w'\|_1 \Rightarrow$$
 results in sparse solutions

ROC Curve - for evaluating logistic classifiers
 - always classifies pos.
 - Bias-Variance Decomposition: equal to assigning $f(w_i) = \exp(-w_i/b)/2b$ Laplacian
 - Fit hypothesis h to X and y where true relationship is g .
 - Bias: inability of hypothesis h to fit g perfectly
 - Variance: error due to fitting to random noise in data
 - Training on more data reduces variance + decreases overfitting



- Our prob. cutoff affects where we fall on the ROC curve
 - Higher area under the ROC curve $\Rightarrow$ better classifier
 - Underfitting: too much bias
 - Overfitting: too much variance

Vector Calculus

$$g = X^T A x \quad \nabla g = (A + A^T) x \quad \nabla^2 g = A + A^T$$

$$g = \|Ax - b\|^2 \quad \nabla g = 2(A^T A x - A^T b) \quad \nabla^2 g = 2A^T A$$

$$g(x) = y^T A x \quad \nabla g = y^T A \quad \nabla^2 g = 0$$

$$g(x) = \|x\|_2^2 \quad \nabla g = 2x \quad \nabla^2 g = 2I$$

- Training error reflects bias, test error reflects both bias + variance
 - Probability $\text{Var}(cX) = c^2 \text{Var}(X)$ Multivariate: $\text{Var}(AX) = A \Sigma A^T$

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$$

 X, Y independent $\Rightarrow E[XY] = E[X]E[Y]$
 $E[X] = \sum P(X=x) x$ or, for continuous $E[X] = \int f(x) x dx$
 Ellipse formula: $x^2 + y^2 = s$ radius = $\sqrt{s}$
 Generative: full probabilistic model of all variables
 Discriminative: provides a model for only the target variables that we want to predict
 e.g. logistic regression

Gradient: $\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$ Hessian: $\nabla^2 f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$

Decision Trees

- Internal nodes test feature values + branch to maximize $H(S) - H_{\text{after}}$ = information gain.

$$H(S) = - \sum p_c \log_2(p_c) \quad \Delta \quad H_{\text{after}} = \frac{|S_L| H(S_L) + |S_R| H(S_R)}{|S_L| + |S_R|}$$

- Decision tree regression: each leaf corresponds to a constant value.

Stopping early strategies - max depth, min samples per node, no info gain (dangerous)

Pruning - growing a large tree; greedily removing splits where removal improves validation performance.

Why stop early? - To reduce overfitting, for computational speed, don't want to fit to noise or overfitting distribution better to estimate posterior probs.

Bagging: bootstrap data, fit many decision trees, have decision trees return average/majority output

Can still result in very similar trees if there are a few features that dominate

Random forests: bootstrap data, fit many DT's. At each split, choose the best split from some randomly selected med features (tends to have higher bias, lower variance than bagging)

Kernels - Used when any weights can be expressed as linear combo of sample points: $w = X^T a$. Strategy is to optimize dual weights instead of primal weights.

Steps:

① Get rid of instances of w by substituting $w = X^T a$. Then rearrange the expression into dual form, which can be written using only a dual weights, $K = XX^T$, and y (labels)

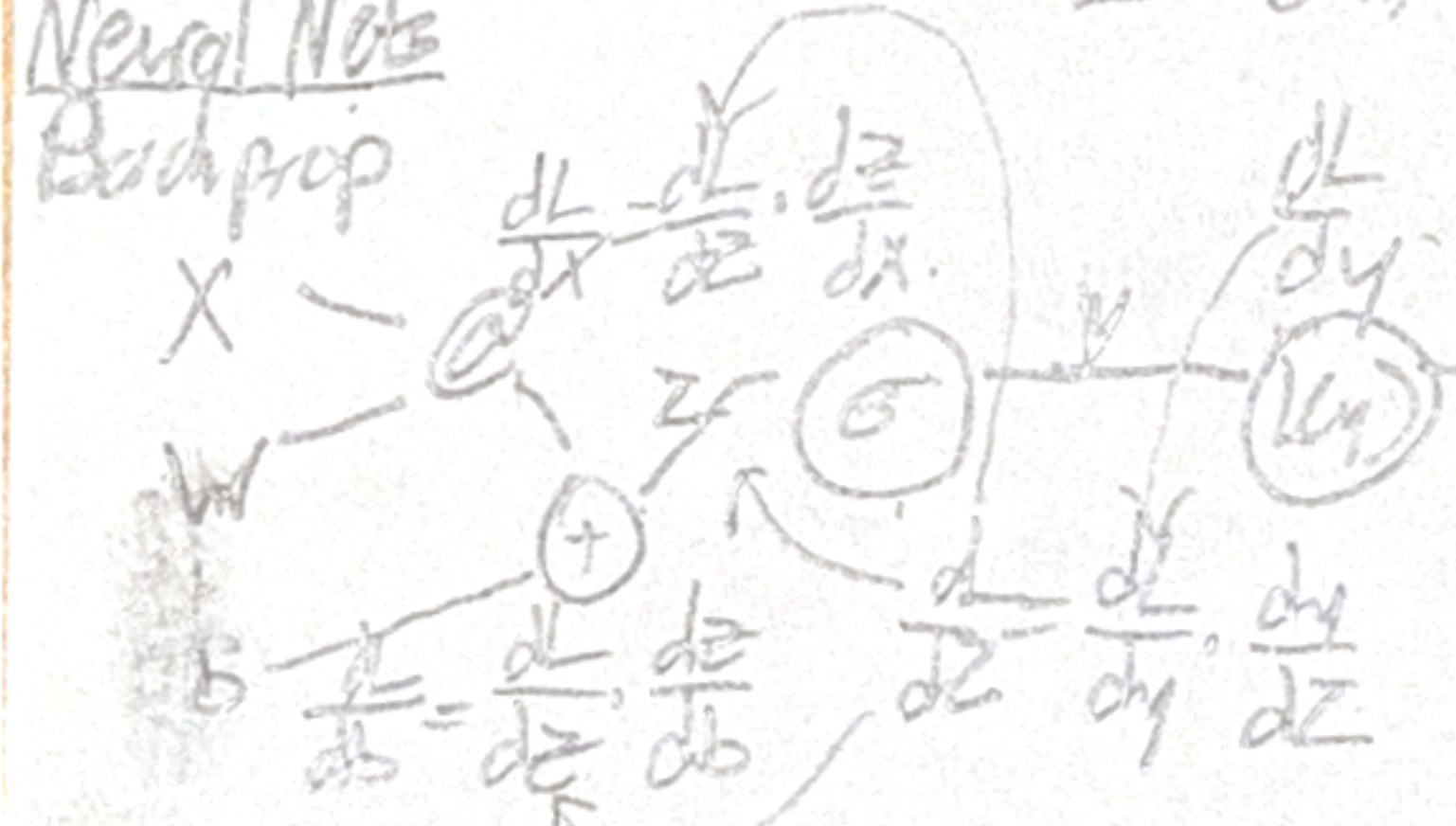
② To classify a point z , compute a weighted sum of kernel functions between z and training point

Kernel matrix:

$$K = \Phi(X) \Phi(X)^T = \begin{bmatrix} k(x_1, x_1) & \dots & k(x_1, x_n) \\ \vdots & & \vdots \\ k(x_n, x_1) & \dots & k(x_n, x_n) \end{bmatrix} \quad K \succeq 0 \Rightarrow K \text{ is a valid kernel infinite input feature space}$$

Neural Nets

Backprop



Regression: usually linear output units

Classification: to choose from $k \geq 3$ classes, use softmax activation

- An NN w/ 1 hidden layer and enough units is a universal function approximator

Vanishing gradient: when sigmoid derivative is flat, learning is slow. Fixes:

- Initialize weights of node w/ indegree n according to $N(0, \frac{1}{\sqrt{n}})$.
- Set labels to 0.9 and 0.1, not 1 and 0
- Modify backprop to add a small constant to s'
- Cross-Entropy loss instead of MSE
- Replace sigmoids w/ ReLUs.

To speed up training, use SGD or normalize the data (gets data in operating region of sigmoid/ReLU)

Heuristics to avoid Overfitting: decrease # units, ensemble of NNs, L_2 regularization (weight decay), L_1 regularization

L_2 Dropout: during training, temporarily disable a random subset of the units. Emulates ensemble, but w/ faster training.

CNNs - BIG IDEA: sliding a 2D kernel across an image and taking the dot product

'Local connectivity': a hidden unit connects only to a small patch of units in the input layer

'Shared weight': groups of hidden units share the same kernel

Formula: $W_{out} = \frac{W_{in} - F + 2P}{S} + 1$ where F = kernel size, P = padding, S = stride

Neurobiology: - Brains are parallel. No neuron death will make you forget something. This is true for NNs too (see dropout)

neuron: cell in brain

action potential/spike: electrochemical impulse fired by neuron to communicate w/ other neurons

axon: the limbs along which the action potential propagates - "output"

dendrite: smaller limb by which neuron receives info - "input"

synapse: connection from one neuron's axon to another dendrite

neurotransmitter: chemical released by axon terminal to stimulate dendrite

Analogies: unit output $\Leftrightarrow$ firing rate of neuron

weight of $\Leftrightarrow$ synaptic strength

positive weight $\Leftrightarrow$ excitatory neurotransmitter

negative weight $\Leftrightarrow$ inhibitory neurotransmitter

linear combo of weights $\Leftrightarrow$ summation

logistic/sigmoid fn $\Leftrightarrow$ firing rate saturation

weight change/learning $\Leftrightarrow$ synaptic plasticity

PCA - dimensionality reduction. often removes dimensions which correspond to noise, which decreases overfitting. Makes computation faster. Like feature subset selection, but not across aligned. MUST CENTER for PCA

- ① Fit Gaussian MLE to data, choose k Gaussian axes of greatest variance
 $\rightarrow$ Center X , (maybe) normalize X , choose k eigenvectors of $X^T X$ w/ the greatest eigenvalues
- ② Find direction w that maximizes the sample variance of the projected data
 $\rightarrow w^* = \underset{w}{\operatorname{argmax}} \frac{w^T X^T X w}{w^T w}$ cluster variation $\rightarrow$ equivalent problem: $\sum_{i=1}^k \frac{1}{h_i} \leq \sum_{j=i}^k \|X_j - \mu_i\|_2^2$
- ③ Find direction w that minimizes mean squared projection distance
 $\rightarrow w^* = \underset{w}{\operatorname{argmin}} \sum_{i=1}^n \left(\|x_i\|^2 - (x_i \cdot \frac{w}{\|w\|})^2 \right)$ K-Means choose k centroids μ_i
 Find y that minimizes $\sum_{i=1}^n \|x_i - \mu_i\|_2^2 \leq \text{NP hard}$
 $\rightarrow$ i.e. minimize sum of squared distances from points to their cluster means

SVD
 $X = U D V^T = \sum_{i=1}^d \sigma_i u_i v_i^T$
 $\begin{matrix} \boxed{X} & = & \boxed{U} & \boxed{\begin{matrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_n \end{matrix}} & \boxed{V^T} \\ n \times d & & n \times d & d \times d & d \times d \end{matrix}$
 v_i is an eigenvector of $X^T X$ with eigenvalue σ_i^2
 u_i is an eigenvector of $X X^T$ with eigenvalue σ_i^2
 Row i of $U D$ gives the principal components of point x_i

Hierarchical Clustering
 Agglomerative: start w/ each point a cluster, repeatedly merge pairs
 Divisive: start w/ all points in one cluster, repeatedly split it.
 Complete linkage: $d(A, B) = \max \{ d(a, b) \mid a \in A, b \in B \}$
 Single link: $d(A, B) = \min \{ d(a, b) \mid a \in A, b \in B \}$
 Average: $d(A, B) = \frac{1}{|A||B|} \sum_{a \in A, b \in B} d(a, b)$
 Centroid: $d(A, B) = d(\mu_A, \mu_B)$
Adaboost (goal: ensemble and reduce bias)
 ① Initialize weights to $w_i \leftarrow \frac{1}{n}$
 ② For $t = 1 \dots T$
 a. Train G_t with weights w_i
 b. Compute weighted error rate $\epsilon_t = \frac{\sum w_i \text{misclassified}}{\sum w_i}$
 $\beta_t \leftarrow \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$
 c. Reweight points $w_i \leftarrow e^{-\beta_t}$ if G_t misclassifies x_i
 ③ Return classifier $h(x) = \operatorname{sign} \left(\sum_{t=1}^T \beta_t G_t(x) \right)$

Learning Theory
 Range space (P, H) where P is the set of all test/training points and H is a set of hypotheses. Each hypothesis is a subset $h \in P$ that specifies which points in P are in class C .
 Dichotomy: $X \cap h \rightarrow$ picks the training points in X that h predicts are in class C .
 Risk $R(h)$ is the probability that h misclassifies a random test point. We estimate $R(h)$ with our training data. Problem: too many hypotheses $\Rightarrow$ one h will get lucky and have a low $R(h)$ but will not generalize well.
 # dichotomies: $\Pi_H(X) = |\{X \cap h : h \in H\}|$ the most of any point set of size n
 Shatter-freen: $\Pi_H(n) = \max_{|X|=n, X \subseteq P} \Pi_H(X) \leq 2^n$
 VC dimension: $VC(H) = \max \{ n : \Pi_H(n) = 2^n \}$
 $\rightarrow$ VC dimension of halfplanes is 3.
 KNN - find k nearest training points for classifying/regressing on a test point
 Cover-Hart: as $n \rightarrow \infty$, KNN error rate $\leq 2B - B^2$ where B = Bayes risk
 As $k \uparrow$, bias $\uparrow$, variance $\downarrow$
 Fix Hodges: as $n \rightarrow \infty$, $k \rightarrow \infty$, $\frac{k}{n} \rightarrow 0$, KNN error rate converges to B .
 2-S dim: use Voronoi diagram's large dim: use exhaustive search, but can use dim. reduction
 5-30 dim: use k -d trees
 k -d query alg: maintains NN found so far v , minleap of updated subtrees, keyed by distance from $q(v)$.
 Can choose a lot of sensing by searching boxes $C(x)$ or array
 Pseudoinverse: $X^+ = V D^+ U^T$
 Adaboost bad at handling outliers due to exponential weights
 Posterior can be estimated with
 $P(Y=1|X) = \frac{1}{1 + e^{-2 \sum \beta_t G_t(x)}}$